

# WhatsApp: lead recovery and a bot that already knows the customer

---

What it takes to (a) message a lead on WhatsApp when they fill the form but never talk to the site chatbot, and (b) run a WhatsApp assistant carrying the full context of everything they typed.

**01**    **The one rule that shapes everything** — the 24-hour window

---

**02**    **Opt-in** — the part that gets numbers banned

---

**03**    **Cloud API or a BSP** — pick your path

---

**04**    **Prerequisites** — the account setup, in order

---

**05**    **Feature A** — the nudge for leads who never chatted

---

**06**    **Feature B** — the WhatsApp bot with form context

---

**07**    **Schema changes** — concrete Prisma

---

**08**    **What it costs**

---

**09**    **Nine things that will bite you**

---

**10**    **Build order**

---

Written against the existing AURUM codebase — Next.js 16, Prisma 7 on Postgres, DeepSeek. Nothing in this document has been implemented. Pricing and platform policy are Meta's and change; verify current figures before committing to a budget.

---

## 01 · The one rule that shapes everything

Almost every surprise in WhatsApp integration traces back to a single policy: **you cannot message someone freely unless they messaged you in the last 24 hours.**

| Situation                                               | What you may send                                                                 |
|---------------------------------------------------------|-----------------------------------------------------------------------------------|
| Customer messaged you < 24h ago<br>the "service window" | Anything. Free-form text, images, buttons. This is also <b>free of charge</b> .   |
| No message, or > 24h ago                                | Only a <b>pre-approved template</b> . Paid. Reopens the 24h window if they reply. |

### This is exactly your situation

A lead who fills the form and never opens the site chat has **never messaged your WhatsApp number**. There is no open window. So the nudge in Feature A **must** be a pre-approved template message — you cannot simply send them a friendly sentence. Every plan that ignores this fails at the first send.

Templates are submitted to Meta and reviewed (usually minutes, sometimes up to 24 hours). They are categorised, and the category sets the price and the rules:

- **Utility** — about a transaction the user initiated: an order, a booking, an enquiry they submitted. Cheaper, approved more readily.
- **Marketing** — promotions, offers, re-engagement, anything persuasive. Most expensive, strictest review, and users can opt out of the whole category.
- **Authentication** — OTPs only. Not relevant here.

Your nudge is a follow-up to an enquiry the customer themselves submitted, so it can legitimately be written as **Utility** — that is worth real money at volume and gets approved far more easily. Write it as a factual follow-up to their enquiry, not as a pitch. The moment it reads promotional ("Special offer on ceramic coating!") it becomes Marketing and may be rejected as the wrong category.

---

## 02 · Opt-in — the part that gets numbers banned

Meta requires **explicit, provable consent** before you send anyone a business-initiated message. This is not a formality: your number carries a quality rating driven by how recipients react. Enough blocks or "report spam" taps and your messaging limit drops, then your number gets restricted.

### What to do

- Add a checkbox to the enquiry form: "*WhatsApp me about this enquiry.*" Unticked by default.
- Store the consent flag, the **timestamp**, and the **page it was given on**. If Meta ever asks, that record is your defence.

- Honour opt-out. If someone replies STOP / "don't message me", set a flag and never send again. Handle this in the webhook.
- Never buy or import numbers. Only people who typed their own number into your form.

#### You already have the hard part

The form normalises every phone number to bare 10 digits before storing it, so +91 98250 41200 and 9825041200 are already one customer. That normalisation is what lets an inbound WhatsApp message — which arrives as 919825041200 — resolve to the right enquiry. Without it this whole feature quietly matches nothing.

## 03 · Cloud API or a BSP

| Path                                                      | Good                                                                                                               | Bad                                                                                                                                                       |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Meta Cloud API</b><br>direct                           | No middleman fee — you pay Meta's per-message rate only. Full control.<br>Hosted by Meta, so nothing to run.       | You build everything: templates, webhook, retries, opt-out, dashboards. Business verification is on you.                                                  |
| <b>BSP</b><br>AiSensy, Interakt,<br>Wati, Gupshup, Twilio | Live in days. Template manager, shared team inbox, broadcast UI, analytics.<br>They walk you through verification. | Monthly fee on top of Meta's per-message cost.<br>Their API, their limits. Bot logic often has to live in their flow builder unless they expose webhooks. |

#### Recommendation

**Cloud API direct.** The deciding factor is Feature B: you want the bot answering from `knowledge.md` with the customer's enquiry in the prompt — the exact system already built and tested. A BSP's visual flow builder actively fights that; you would end up using them as a dumb pipe while still paying the monthly fee. The one reason to pick a BSP is if the studio needs a **human team inbox** to take over conversations, which is genuinely painful to build yourself.

## 04 · Prerequisites, in order

1

### Meta Business Account

business.facebook.com. Complete

#### Business Verification

— documents proving the business exists. Start here; it is the slowest step and everything else waits on it.

2

### WhatsApp Business Account (WABA)

Created inside Meta Business Suite and linked to a Meta app in developers.facebook.com.

3

### A dedicated phone number

It

cannot

be active on the normal WhatsApp or WhatsApp Business app. If the studio's number is already on WhatsApp you must delete that account first, or buy a new number. Decide this early — it is the most common blocker.

#### 4 Display name approval

"AURUM Detail Studio" must match the verified business name or it gets rejected.

#### 5 Permanent access token

Create a System User in Business Settings and issue a token with `whatsapp_business_messaging` and `whatsapp_business_management`. The token from the dev console expires in 24 hours — do not build on it.

#### 6 A public HTTPS webhook

Meta must reach your server. Localhost will not do — use the deployed URL, or `ngrok` while developing.

You end up with four secrets:

```
WHATSAPP_TOKEN=           # permanent system-user token
WHATSAPP_PHONE_NUMBER_ID=  # the sending number's id, not the number
WHATSAPP_WABA_ID=
WHATSAPP_VERIFY_TOKEN=    # any random string you invent; Meta echoes it back
WHATSAPP_APP_SECRET=      # to verify webhook signatures
```

---

## 05 • Feature A — the nudge

**Goal:** someone submits the form, never sends a message in the site chat, and a while later gets a WhatsApp from the studio.

### Defining "never talked to the chatbot"

Precisely: the customer has an `Enquiry`, and their `Conversation` contains **zero messages with role USER**. The assistant's own opening does not count — the test is whether the human said anything back. That is a single query against the existing schema.

### Why it needs a scheduler

You cannot decide at submit time; you have to wait and see whether they engage. So the flow is: on submit, write a **scheduled outbox row** due in ~20–30 minutes. A worker wakes up, re-checks the condition, and sends only if it still holds. Options: Vercel Cron hitting a protected route every 5 minutes, a Postgres-backed queue polled by a worker, or a proper queue (BullMQ + Redis). For this volume a cron route reading an outbox table is the honest choice.

### The template

Submitted once, approved, then filled with variables per send:

Hi {{1}}, this is AURUM Detail Studio in Ahmedabad.

We have your enquiry {{2}} about {{3}} for your {{4}}.

Reply here and I can give you the price band and what the inspection involves. If now is not a good time, reply STOP and we will not message again.

**UTILITY** — it references an enquiry they submitted, states a fact, and offers to continue. No offer, no urgency, no adjectives. That is what gets a Utility approval rather than a Marketing rejection.

## Sending it

POST [https://graph.facebook.com/v21.0/{PHONE\\_NUMBER\\_ID}/messages](https://graph.facebook.com/v21.0/{PHONE_NUMBER_ID}/messages)  
Authorization: Bearer {WHATSAPP\_TOKEN}

```
{
  "messaging_product": "whatsapp",
  "to": "919825041200",
  "type": "template",
  "template": {
    "name": "enquiry_follow_up",
    "language": { "code": "en" },
    "components": [{
      "type": "body",
      "parameters": [
        { "type": "text", "text": "Rohan" },
        { "type": "text", "text": "AUR-7Q2K4M" },
        { "type": "text", "text": "ceramic coating" },
        { "type": "text", "text": "2022 Hyundai Creta" }
      ]
    }]
  }
}
```

### Make it idempotent

Store the returned message id and mark the outbox row sent **in the same transaction** as the send. A cron that retries a half-finished row will message the same customer twice, which is precisely the behaviour that destroys a number's quality rating.

---

## 06 • Feature B — the WhatsApp bot with full context

This is the cheaper half, because the thinking is already done. The site chatbot's system prompt is assembled by `buildSystemPrompt()` from `knowledge.md` plus the customer's enquiry. WhatsApp does not need a second brain — it needs a second **transport**.

### The flow

#### 1 Meta POSTs your webhook

An inbound message arrives with the sender's number as `919825041200`.

## 2 Verify the signature

Check `X-Hub-Signature-256` against your app secret, then

**respond 200 immediately**

and do the work after. Meta retries anything slow, and a retry means a duplicate reply.

## 3 Resolve the customer

Strip the `91` country code to the bare 10 digits your database already stores, and look up `Customer`.

Found → load their latest enquiry and conversation. Not found → an unknown number; either reply with a generic greeting or hand to a human.

## 4 Build the prompt — unchanged

Call the existing `buildSystemPrompt({ enquiry, prior })`. The bot now knows their name, car, segment, chosen service, what they typed, and every earlier message, on WhatsApp, without asking a thing.

## 5 Call DeepSeek — but not streamed

WhatsApp has no partial messages. Drop `stream: true` and take the whole reply. Optionally send a typing indicator first.

## 6 Persist, then send

Write both messages to the existing `Message` table tagged `channel: WHATSAPP`, then POST the reply.

One transcript per customer across both channels — the studio sees one conversation, not two.

### The payoff of doing it this way

A customer can start on the website, drop off, get the nudge, and continue on WhatsApp — and the assistant picks up mid-thread because it is reading the same `Conversation` rows. That continuity is the whole point, and it only works because the phone number was normalised at the door.

## Free-form replies are allowed here

Once they reply to your template, the 24-hour window opens and the bot can answer normally — no templates, no per-message charge for those service messages. If a conversation goes quiet for over 24 hours, you are back to needing a template to restart it.

---

## 07 · Schema changes

```
model Customer {
  // ... existing fields
  whatsappOptIn    Boolean    @default(false)
  whatsappOptInAt  DateTime?
  whatsappOptInFrom String?    // page the consent was given on
  whatsappBlocked  Boolean    @default(false) // they replied STOP
}

model Message {
  // ... existing fields
  channel    MessageChannel @default(WEB)
  externalId String?        @unique // Meta's message id — dedupes retries
}
```

```

}

enum MessageChannel { WEB WHATSAPP }

/// Scheduled outbound WhatsApp. A row is a promise to send later,
/// re-checked at send time in case they engaged in the meantime.
model WhatsappOutbox {
  id          String    @id @default(cuid())
  customerId   String
  enquiryId    String
  templateName String
  payload      Json
  dueAt        DateTime
  sentAt       DateTime?
  failedAt     DateTime?
  attempts     Int       @default(0)
  lastError    String?
  externalId   String?

  @@index([dueAt, sentAt])
}

```

## Files this adds

| File                                            | Does                                                                                              |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>src/lib/whatsapp.ts</code>                | Cloud API client: <code>sendTemplate()</code> , <code>sendText()</code> , signature verification. |
| <code>src/app/api/whatsapp/route.ts</code>      | GET verifies the webhook, POST receives messages.                                                 |
| <code>src/app/api/cron/whatsapp/route.ts</code> | Drains the outbox. Protected by a secret header.                                                  |
| <code>src/lib/deepseek.ts</code>                | Add a non-streaming <code>complete()</code> beside the streaming path.                            |
| <code>enquiry/route.ts</code>                   | Write the outbox row when consent was given.                                                      |

## 08 • What it costs

| Item                                     | Roughly                                           |
|------------------------------------------|---------------------------------------------------|
| Cloud API access                         | Free                                              |
| Service messages (inside the 24h window) | Free                                              |
| Utility template (India)                 | Fractions of a rupee to ~₹0.2–0.4 per message     |
| Marketing template (India)               | Several times the utility rate — roughly ₹0.7–1.0 |
| BSP platform fee, if used                | ~₹1,000–3,000 / month                             |
| DeepSeek per reply                       | Negligible — well under ₹0.5 per conversation     |

Meta has moved toward per-message pricing and periodically revises rates and free allowances by country. Treat the above as orders of magnitude and check the current India rate card before budgeting. At nine cars a week the total is small either way; the platform fee, not the messages, is the real cost.

---

## 09 · Nine things that will bite you

1. **The 24-hour window.** Half of all "why won't it send" is this.
2. **The number is already on WhatsApp.** Discover this before you plan a launch date, not during one.
3. **Template rejection.** Usually wrong category, or variables at the very start/end of the body. Keep a plain sentence around each `{{n}}`.
4. **Duplicate webhooks.** Meta delivers at-least-once. Dedupe on the message id — that is what `Message.externalId @unique` is for.
5. **Slow webhook responses.** Return 200 in milliseconds and do the AI call after, or Meta retries and your customer gets two replies.
6. **Quality rating.** Messaging limits start low and tier up on good behaviour. Blocks push you the other way, fast.
7. **No streaming.** The typewriter effect has no WhatsApp equivalent. Send one finished message.
8. **Opt-out is not optional.** Detect STOP and stop, permanently.
9. **Business verification takes real time.** Days, occasionally longer. It is the long pole — start it first.

---

## 10 · Build order

- 1 **Paperwork, in parallel with everything**  
Business verification, WABA, sort out the phone number. Blocks all sends; blocks no code.
- 2 **Consent**  
Checkbox on the form, four columns on `Customer`. Ship it early so real consent accumulates while the rest is built.
- 3 **Feature B before Feature A**  
Counter-intuitive but correct: the inbound bot can be tested by messaging your own number, needs no templates and no approvals, and reuses the prompt builder you already trust. Get the loop working end to end here.
- 4 **Submit the template**  
Do it as soon as the WABA exists — approval is a waiting game you can overlap with development.
- 5 **Feature A last**  
Outbox table, cron route, send. Test with the studio's own numbers before it touches a real lead.

Realistically: a few days of engineering spread across one to two weeks of waiting on Meta. The waiting dominates, which is the argument for starting the verification today and writing the code around it.



---

AURUM Detail Studio is a fictional business built to showcase a marketing site. This document describes work that has **not** been implemented.